**Đại học Quốc gia Thành phố Hồ Chí Minh**

**Đại học Khoa học tự nhiên**

*Khoa Công nghệ thông tin*


# HOMEWORK
## DOMAIN TESTING

Subject SOFTWARE TESTING
Class 22KTPM3
Student **22127374 – Lê Thanh Tâm**

*Ho Chi Minh City, 2025*

# Table of Contents

# 1 Group Task Allocation

| Fullname | StudentID | Feature |
|---|---|---|
| Lương Hoàng Dung | 22127076 | 1. User - Product Browsing Experience<br>2. User - Contact Form Submission<br>3. User - Checkout |
| Lê Thanh Tâm | 22127374 | 1. Admin - Orders<br>2. Admin - Categories<br>3. Admin - Products |
| Nguyễn Lâm Nhã Uyên | 22127445 | 1. Log in<br>2. Sign up |
| Lê Nguyễn Yến Vy | 22127466 | 1. User - add to cart and quantity management<br>2. User - change password<br>3. User - view and manage invoices<br>4. Admin - manage and reply to user's messages |

# 2 Introduction

## 2.1 Selected Features for Testing

The following administrative features were selected to be tested using the **Domain Testing** methodology due to the input fields and complex business rules:

- **Admin - Orders:** To test search domains, filters, and status logic.
- **Admin - Categories:** To test data constraints (name, slug) and hierarchical logic.
- **Admin - Products:** To apply in-depth EP and BVA techniques across its many diverse input fields.

## 2.2 Testing Methodology

The primary methodology was **Domain Testing**, which focuses on analyzing input data domains to create effective test cases. This was achieved using two core techniques:

- **Equivalence Partitioning (EP):** To divide data into logical classes (e.g., valid, invalid) and reduce the number of test cases.
- **Boundary Value Analysis (BVA):** To test the values at the edges of these classes (e.g., min/max values), where errors are most common.

# 3 Feature 1: Admin - Orders

## 3.1 Feature Description

The **Admin - Orders** feature provides a comprehensive interface within the Admin panel for managing and processing orders. It enables administrators to search, view, and update order details efficiently while ensuring robust validation and security measures. Key functionalities include:

- **Search Capabilities**: Supports searching orders by invoice number (valid/existing or non-existing), billing address (partial or exact matches), invoice date (specific dates, current date, or full datetime strings), and status filters via a dropdown menu. Handles edge cases such as empty strings, very short or long search terms (up to 255 characters), special characters, case sensitivity, and leading/trailing whitespace.
- **Order Status Management**: Enforces a sequential order status workflow (AWAITING_FULFILLMENT → ON_HOLD → AWAITING_SHIPMENT → SHIPPED → COMPLETED), preventing reversion to previous statuses or non-sequential updates. Supports manual status updates via a dropdown menu, automatic status updates upon successful customer payment, and validation to ensure only valid status transitions are allowed.
- **Status Message Handling**: Allows administrators to save meaningful status messages (up to 255 characters) with robust input validation. Rejects or sanitizes special characters and HTML/script tags to prevent Cross-Site Scripting (XSS) vulnerabilities. Handles edge cases like empty inputs, whitespace-only inputs (treated as empty after trimming), and messages at or beyond the maximum length limit.
- **Data Integrity and Security**: Ensures non-editable fields (e.g., Invoice Number, Total, Billing Address) remain protected during order edits. Validates inputs to prevent invalid data (e.g., incorrect tracking numbers) and maintains synchronization between Payment and Order Management subsystems for accurate status updates.
- **User Interface and Functionality**: Provides navigation to the order edit interface, a reset button to clear search filters, and error handling for invalid inputs or actions (e.g., attempting to update to an unchanged status). Ensures all interactions are secure, user-friendly, and compliant with system constraints.

This feature is critical for streamlined order management, ensuring data accuracy, secure operations, and seamless integration with payment processing, as validated through 30 test cases covering search, status updates, input validation, and system synchronization.

## 3.2 EP and BVA Design Process

### 3.2.1 Inputs and Constraints:

This category does not test broad data classes (like EP) or edges (like BVA), but instead focuses on **specific business rules** and the **core functionalities** of the system. These are the "rules of the game"that the system must follow.

- **Business Rule Examples (TC21, TC23, TC24):**
  - These are business logic constraints. **TC23** tests the rule "Cannot revert to a previous status."
  - **TC24** tests the data synchronization constraint between the payment system and the order system. They are rules, not classes of data.
- **Core Functionality Examples (TC16, TC18):**
  - Testing the "Reset"and "Edit"buttons ensures that the page's basic, essential functions work correctly. This is testing a functional constraint: "The Reset button must clear the filter."

#### 3.2.2 Equivalence Partitioning (EP):

EP is used to test different **classes of search data** and various **status update actions**, rather than attempting to test every single invoice or date.

- **Examples (TC01, TC02, TC17):**
  - `TC01` represents the **entire group** of valid invoices existing in the system.
  - `TC02` represents the **infinite group** of correctly formatted but non-existent invoices.
  - `TC17` represents the **action** of a successful update between any two valid statuses.

#### 3.2.3 Boundary Value Analysis (BVA):

BVA focuses on the **limits (boundaries)** of the input fields, especially **string length**, as this is where errors are most likely to occur.

- **Example (TC06, TC07):**
  - Assuming the search field is limited to 255 characters, we test right **at the boundary (255 chars)** and right **over the boundary (256 chars)** to "hunt"for off-by-one errors in conditions (< instead of <=).

### 3.3 Test Case Documentation

As an exercise in Domain Testing, a comprehensive set of test cases was designed for the **Admin - Orders** feature. These test cases primarily focus on applying two techniques:

- Equivalence Partitioning (EP) to test various data classes (e.g., searches with valid, non-existent, or special character inputs).
- Boundary Value Analysis (BVA) to test the limits of input fields (e.g., the maximum length of search strings and status messages).

The detailed documentation for each test case—including the Test Case ID, Description, Steps, and the specific application of EP/BVA—has been carefully recorded. For a complete and precise understanding, please access and review the Test cases sheet in the file `22127374_Test Cases.xlsx`.

## 4 Feature 2: Admin - Categories

### 4.1 Feature Description

The **Admin - Categories** feature enables management of product categories with:

- **Add/Edit:** Create/edit top-level/sub-categories with valid Name (up to 120 characters), Slug, and Parent ID. Rejects blank/duplicate Names, emojis, invalid Slugs, or Names over 120 characters. ID is display-only.
- **Delete:** Delete leaf-node/top-level categories not used as Parent IDs; blocks deletion of parent categories.

- **Search:** Search by full/partial Name, single character, or emoji; handles non-matching strings and resets via Reset button.
- **Navigation:** Back button discards changes on Add/Edit forms, returning to the Categories list.
- **Frontend Display:** Top-level categories (e.g., Hand Tools) appear in navigation; clicking leads to correct pages with accurate sub-category filters.
- **Sorting/Filtering:** Sort products by Name (A-Z/Z-A) or Price (High-Low/Low-High) on Hand Tools/Power Tools pages. Filter by sub-category (e.g., Pliers, Sander), brand, or both; clear filters to reset. Ensures efficient category management and accurate product organization.

## 4.2 EP and BVA Design Process

### 4.2.1 Inputs and Constraints:

This is where the **operational rules** of category management and the **critical functional interactions** on the page are tested.

- **Deletion Rule Example (TC23):**
  - **TC23** tests a critical data integrity constraint: "A category that is a parent cannot be deleted."This is a hard business rule.
- **Sorting/Filtering Functionality Examples (TC33-TC38):**
  - This group of test cases checks if the Sort and Filter functions meet their requirements. For example, **TC33** tests the constraint "When Sort by Name A-Z is selected, the list must be ordered correctly."This is functional testing, not data input testing.

### 4.2.2 Equivalence Partitioning (EP):

EP helps test different **types of categories** (parent, child) and **data constraint violation scenarios** (duplicate, invalid format) without needing to create hundreds of categories.

- **Examples (TC01, TC02, TC12):**
  - **TC01** represents the **group** of all parent categories.
  - **TC02** represents the **group** of all sub-categories.
  - **TC12** represents the **group** of all attempts to create a category with a duplicate name.

### 4.2.3 Boundary Value Analysis (BVA):

The most important constraint here is the **length of the Category Name**. BVA focuses on testing the **edges** of this constraint.

- **Example (TC05, TC06, TC08):**
  - To ensure the condition `1 <= length <= 120` is programmed correctly, we test the boundary values of `1`, `120`, and `121`.

## 4.3 Test Case Documentation

- The test cases for the **Admin - Categories** feature were built focus on the principles of Domain Testing. The Boundary Value Analysis (BVA) technique was particularly emphasized to test the length constraint of the Category Name—a critical data domain. In parallel, Equivalence Partitioning (EP) was used to validate scenarios like creating parent/child categories and handling duplicate data constraints.
- The detailed documentation for each test case—including the Test Case ID, Description, Steps, and the specific application of EP/BVA—has been carefully recorded. For a complete and precise understanding, please access and review the Test cases sheet in the file `22127374_Test Cases.xlsx`.

# 5 Feature 3: Admin - Products

## 5.1 Feature Description

The **Admin - Products** feature enables administrators to manage products with the following functionalities:

- **Search:** Search by full/partial product name or stock quantity, handling empty strings, spaces, or long inputs. Displays "No products found"for no matches.
- **Add Product:** Add products with mandatory (Name, Price, Brand) and optional (Description, Category) fields. Supports names up to 220 characters, valid numeric Price/Stock, and handles invalid inputs (e.g., blank Name, non-numeric Price).
- **Edit Product:** Update fields with valid data, allow blank Description, prevent editing ID or invalid inputs, and permit duplicate names.
- **Delete Product:** Supports soft delete with confirmation, prevents deletion of products with dependencies (e.g., in orders), and allows canceling deletion.
- **Pagination:** Navigate via Next, Previous, or specific page numbers, maintaining state after searches, with correct first/last page display.
- **Reset/Back:** Reset button restores the product list; Back button discards changes or returns to the list.

Ensures efficient product management, input validation, and user-friendly pagination.

## 5.2 EP and BVA Design Process

### 5.2.1 Inputs and Constraints:

This category verifies **required data constraints**, **logical rules** for deleting/editing, and **complex functionalities**.

- **Required Data Example (TC10, TC19):**

  - **TC10** tests the constraint "Product Name cannot be empty."This is a basic rule about data validity, a hard constraint.

- **Logical Rule Example (TC42):**
  - **TC42** tests the data integrity constraint: "A product that is already part of an order cannot be deleted"to protect historical data.

- **Complex Functionality Example (TC32):**
  - **TC32** tests the interaction between two features: Pagination and Search. The constraint is: "Pagination must work correctly on the results filtered by a search."

#### 5.2.2 Equivalence Partitioning (EP):

The Product feature has many input fields, making EP essential for testing the **different classes of data** for each field (text, numbers, currency, null, etc.).

- **Examples (TC17, TC21):**
  - **TC17** (Enter text into Stock) represents the **group** of all non-numeric inputs.
  - **TC21** (Enter price with 3 decimal places) represents the **group** of scenarios that violate the "currency format"constraint.

#### 5.2.3 Boundary Value Analysis (BVA):

BVA is used to check the **limits** of critical fields like **Product Name length**, the **value of the Price**, and the **behavior of Pagination**.

- **Examples (TC22, TC30, TC31):**
  - **TC22** tests the **boundary value** of the "Price"data type in the database.
  - **TC30** and **TC31** test the **boundary states** of the pagination component (the first and last page).

### 5.3 Test Case Documentation

Testing the **Admin - Products** feature served as an in-depth exercise in Domain Testing. The feature's numerous input data domains provided an excellent opportunity to broadly apply:

- Equivalence Partitioning (EP) to test different data types for the Stock and Price fields (numeric, text, empty).
- Boundary Value Analysis (BVA) to test critical values such as the maximum value for Price, the length limits of the Product Name, and the boundary states of the pagination feature.

The detailed documentation for each test case—including the Test Case ID, Description, Steps, and the specific application of EP/BVA—has been carefully recorded. For a complete and precise understanding, please access and review the Test cases sheet in the file `22127374_Test Cases.xlsx`.

# 6 Merged Test Case List

The final test suite was developed through a hybrid process of AI generation and human oversight to maximize both efficiency and testing depth.

## 6.1 Consolidation of AI and Manual Test Cases

The process began by merging two distinct sets of test cases:

- **AI-Generated Baseline:** This provided broad and rapid coverage of standard scenarios based on **Equivalence Partitioning (EP)** and **Boundary Value Analysis (BVA)**, such as data type checks and input length limits.
- **Manual Augmentation:** This added depth and context, focusing on complex, multi-step business scenarios and exploratory test cases that require human intuition and domain expertise.

## 6.2 Deduplication Process and Justification

The consolidated list was then refined to eliminate redundancy. While some test cases with similar objectives were merged, others that appeared alike were intentionally kept separate for precise validation.

- **Key Justification Example:** The test case for **searching with a maximum-length string (TC06)** and the one for **saving a maximum-length status message (TC27)** were both kept.
  - **Reasoning:** Although both apply the same BVA principle, they validate two **completely independent functional components** (the search function vs. the status update function). A failure in one does not imply a failure in the other. Therefore, testing them separately is critical to ensure the integrity of each individual function. The final decision on merging or retaining test cases was always made by the human tester to ensure thorough coverage was not compromised.

# 7 Use of AI Tools

## 7.1 AI Tool Description

To leverage state-of-the-art technology, the project utilized the most current and powerful versions of leading Large Language Models (LLMs) available at the time of the testing phase. The specific flagship models were:

- **Gemini 2.5 Pro (Google):** The most advanced and capable version in the Gemini family publicly released by Google, known for its large context window and multimodal reasoning.
- **GPT-4o (OpenAI):** OpenAI's latest and most advanced flagship model ("o"for "omni"), which powers the premium ChatGPT service and surpasses previous iterations like GPT-4.
- **Grok-1.5 (xAI):** The most powerful and recent version of the Grok model that has been officially released and documented by xAI.

These specific versions were chosen to ensure the project benefited from the most up-to-date capabilities in logical analysis, data generation, and scenario brainstorming.

## 7.2 Prompts Used

A multi-stage prompting strategy was used to break down the task, moving from broad generation to detailed analysis. Below are representative examples based on the **Admin - Orders** feature and the expected AI responses.

### Prompt 1: Initial Broad Test Case Generation

> "Based on the following feature description for **Admin - Orders**: 'Manages orders in the Admin panel, enabling search, view, and status updates. Supports searching by invoice number, billing address, invoice date, or status; validates inputs (special characters, length, whitespace); ensures sequential status updates.
>
> Design a comprehensive set of test cases using Domain Testing. Focus on Equivalence Partitioning (EP) for the search field (valid, non-existent, special characters) and Boundary Value Analysis (BVA) for its length (min, max, over max). Present in a table."

**AI Results:**

| Test Case ID | Description | Test Steps | Expected Result |
|---|---|---|---|
| (TC01) | EP: Search with a valid and existing Invoice Number. | 1. Navigate to Orders. 2. Enter 'INV-2022000002' in search. 3. Click Search. | The list updates to show only the order matching 'INV-2022000002'. |
| (TC02) | EP: Search with a valid but non-existing Invoice Number. | 1. Navigate to Orders. 2. Enter 'INV-NONEXISTENT123' in search. 3. Click Search. | The list is empty and shows a "No orders found"message. |
| (TC06) | BVA: Search using a string at the maximum length limit (255 chars). | 1. Navigate to Orders. 2. Enter a 255-character string. 3. Click Search. | The system handles the input without errors and returns search results (likely none). |

### Prompt 2: Deep-Dive into a Specific Business Logic

> "Thank you. Now, let's focus specifically on the order status update logic. The required sequence is: AWAITING_FULFILLMENT → ON_HOLD → AWAITING_SHIPMENT → SHIPPED → COMPLETED."
>
> Please design detailed test cases to verify:
>
> - Successful sequential update scenarios.
> - Scenarios that attempt to skip a step.
> - Scenarios that attempt to revert to a previous status."

**AI Results:**

- (TC20) - Sequential Update: Design a test to update an order through each consecutive status from AWAITING_FULFILLMENT to COMPLETED, verifying success at each step.
- (TC21) - Non-sequential Update: Design a test to directly update an order from AWAITING_FULFILLM to SHIPPED, skipping intermediate steps, to check if the system allows or blocks this action.
- (TC23) - Reverting Status: Design a test where an order is already SHIPPED, and attempt to change its status back to ON_HOLD to verify that the system prevents reverting to a previous status.

**Prompt 3: Generating Specific Test Data**

> "For the test case that checks for handling extra whitespace in the search field (TC10), I need some test data. Please generate 5 example search strings for the 'Billing Address' field that contain extra whitespace."

**AI Results:**

- "Test street 123"(Leading whitespace)
- "Test street 123 "(Trailing whitespace)
- "Test street 123 "(Both leading and trailing)
- "Test street 123"(Excessive spaces between words)
- "Midway Road "(Another example with both)

**Prompt 4: Final Review for Redundancy and Gaps**

> "This is my final consolidated list of test cases for the 'Admin - Orders' feature. Please review it and answer two questions: 1. Are there any test cases with overlapping objectives? 2. Based on the initial feature description, are there any coverage gaps?"

**AI Results:**

- **Redundancy Check**: TC06 (Search max length) and TC27 (Status message max length) test the same BVA principle on different fields. They are not redundant as they validate separate inputs, but ensure their titles clearly state which field is being tested.
- **Coverage Gap Analysis**: There appears to be a gap. The current tests verify searching by status (TC11) and by date (TC12) independently. I recommend adding a new test case: 'Search with a valid combination of Status and Invoice Date' to ensure multi-criteria filtering works as expected.

## 7.3 Validation and Refinement Process

A systematic, four-step process was implemented to ensure the quality and relevance of the final test suite.

- **Foundational Review**: The initial test cases generated by the AI were treated as a "first draft."They were immediately reviewed by the tester to correct any misinterpretations of the requirements and to align them with the project's specific terminology and context.
- **Human-led Augmentation**: Recognizing that an AI may not capture all business nuances, the initial set was expanded with manually created test cases. These new cases focused on complex, multi-step user scenarios, specific business rules not explicitly documented, and exploratory tests based on the tester's intuition and domain experience.
- **AI-Assisted Consolidation**: The combined list (AI-generated + manual) was then fed back to the AI for a final review using the second prompt. The AI's suggestions for eliminating redundancies were critically assessed. For example, if the AI suggested merging a test for an "empty string"and a "whitespace-only string,"the tester made the final call on whether this was appropriate or if they needed to remain separate tests to check the trim() function.
- **Final Approval**: The fully refined and consolidated list was then formally approved by the tester, who holds ultimate responsibility for the scope and quality of the test suite.

## 7.4 Breakdown of AI vs. Manual Test Cases

The final test suite represents a hybrid model of AI and human collaboration, with the following approximate breakdown:

- **AI-Generated Baseline ( 60%)**: The majority of the test cases originated from the AI's initial output. These primarily covered the standard, systematic application of Equivalence Partitioning and Boundary Value Analysis for individual input fields (e.g., correct/incorrect data types, formats, min/max length boundaries). This provided a broad and solid foundation very quickly.
- **Manual-Created & Heavily-Modified ( 40%)**: The remaining portion of the test cases were either created from scratch or significantly modified by the tester to add depth and context. This portion includes:
  - **Complex Scenarios & Business Logic ( 25%)**: Multi-step user journeys and tests for specific business rules that require a deeper understanding of the system's goals (e.g., "a parent category cannot be deleted").
  - **Exploratory & Context-Specific Tests ( 15%)**: Test cases based on the tester's intuition about potential user confusion, non-obvious workflows, or specific integration points.

# 8 Test Execution and Bug Reporting

## 8.1 Test Execution Summary

- The test execution phase was conducted from **June 3, 2025, to June 5, 2025**.

- All test cases designed in the documentation suite were executed manually on the project. The execution scope covered the three main features under review: Admin - Orders, Admin - Categories, and Admin - Products. Each test case was performed following the documented steps, and the actual results were compared against the expected results.
- The execution process revealed a significant number of failures. For every test case with a "FAILED"status, a corresponding defect was logged in the bug tracking system to ensure proper monitoring, prioritization, and resolution. A detailed analysis of these defects is presented in the following section.

## 8.2 Bug Report and Analysis

### Defect Summary

The testing process has identified a significant number of defects across all three features. A large portion of these defects are classified with **High** priority, indicating that the quality of the current build is below the acceptable threshold for release.
These are not minor UI issues, but serious flaws affecting **core business workflows**, **data integrity**, and **essential administrative functionalities**. Many critical features, such as product searching, sorting, filtering, and data validation, are either non-operational or fundamentally broken. This situation poses a significant risk to business operations and could lead to incorrect data and a poor user experience for administrators.
Fixing these defects, especially those with **High** priority, must be considered the top task for the development team before any further regression testing or release planning can proceed.

### Highlighted High-Priority Defects

While a comprehensive list is available in the bug report file, the following defects are highlighted due to their severe impact on the system's functionality and integrity.

- **BUG-017 (Priority: High) - Order Status Fails to Synchronize After Payment**
  - **Description:** The order status in the admin panel does not automatically update after a customer's payment is successfully confirmed.
  - **Impact:** This breaks the core automated order processing workflow, requiring manual intervention and creating a high risk of overlooking paid orders.
- **BUG-025 (Priority: High) - Product Sorting and Filtering on Category Pages are Non-Operational**
  - **Description:** On the public-facing category pages, the functionalities to sort products (by Name, by Price) and to filter them are completely broken.
  - **Impact:** This severely degrades the customer's shopping experience, making it impossible for them to find products efficiently.
- **BUG-041 (Priority: High) - Product Search Functionality is Completely Broken**
  - **Description:** The search bar on the "Admin - Products"page consistently fails to return any results, rendering a fundamental administrative tool useless.

  - **Impact:** This makes it impossible for admins to manage the product catalog efficiently.

- **BUG-045 (Priority: High) - System-Wide Form Validation Vulnerabilities**
  - **Description:** The "Add/Edit Product"and "Add/Edit Category"forms have critical gaps in data validation, allowing the submission of incorrectly formatted data.
  - **Impact:** This poses a major threat to the system's data integrity, which will lead to further bugs and operational issues.

These are just a few representative examples of the high-priority issues found. **For a complete and detailed list of all defects, including their specific reproduction steps, priority, and current status, please refer to the file** `22127374_Test Report.xlsx`**.**

# 9 Self-Assessment

| Criteria | Description | Max Points | Self-Assessed Points |
|---|---|---|---|
| Feature Selection | 2 important features selected | 1.0 | 1.0 |
| EP Technique | Correct and complete partition identification | 2.0 | 2.0 |
| BVA Technique | Correct identification of boundaries and rationale | 1.0 | 1.0 |
| Test Case Design | Test cases are clear, traceable, professional | 2.0 | 2.0 |
| Use of AI Tools | Prompt transparency, critical validation, added value | 1.0 | 1.0 |
| Test Execution | All designed test cases executed, results logged | 1.0 | 1.0 |
| Bug Reporting | Clear and complete bug report(s), if applicable | 1.0 | 1.0 |
| Merging and Final Review | Proper combination and deduplication of test cases | 0.5 | 0.5 |
| Presentation | Clarity | 0.5 | 0.5 |
| Total | | 10.0 | 10.0 |